



**POLYTECHNIQUE
MONTREAL**

UNIVERSITÉ
D'INGÉNIERIE

Département de Génie Informatique et Génie Logiciel
LOG6953FE: Digital Twin Engineering
Winter 2025

Digital twins for industry 4.0

Project report

Instructor:

Dr. Bentley Oakes

Team:

Talos Robotics

Team Members:

Hongjian Huo

Ali Entezari

Dmytro Humeniuk

Plan

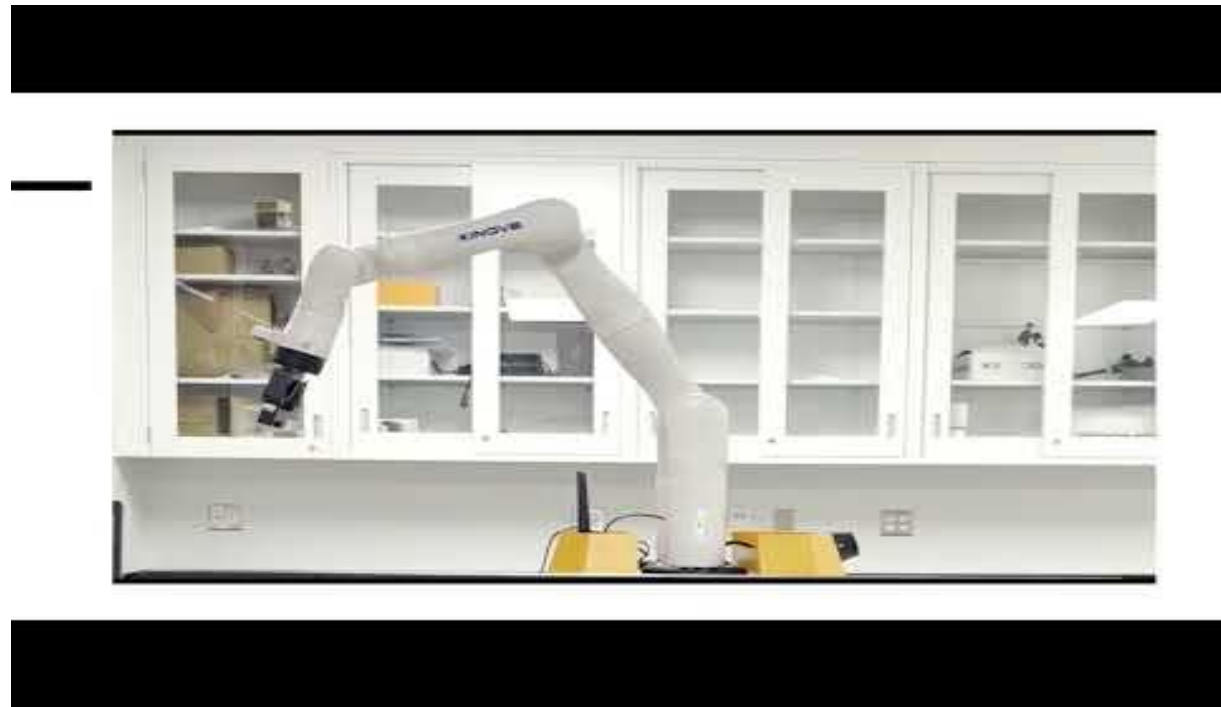
Abstract	3
Section 1 - Introduction	4
Section 2 - Physical twin description	6
2.1 Physical twin overview	6
2.2 Physical twin implementation	8
2.3 Implementation challenges	9
Section 3 - Digital twin architecture and services	12
3.1 Digital twin overview	12
3.2 DT services implementation description	13
3.2.1 Simulation service	16
3.2.2 Singularity detection service	19
3.2.3 Visualization service	20
3.2.4 Deviation monitoring service	21
3.2.5 Emergency stop	22
3.4 Digital to physical twin communication	23
Section 4 - Conclusions	26
Appendix A - Digital twin characteristics table	28

Digital Twins for Industry 4.0

Abstract

An autonomous robotic manipulator accomplishes a pick-and-place task in an industrial environment. It is critical to ensure that the robot is functioning as expected at all times. Therefore, we propose a lightweight digital twin (DT) that allows the operator to monitor the robot using an intuitive graphical user interface (GUI). It alerts operators in case of unexpected behavior or takes actions autonomously to ensure a safe shutdown. Moreover, we develop a service that allows us to simulate the behavior of the robot with minimal deviation from reality and perform diverse “what-if” analysis.

Link to the video: <https://youtu.be/xF0SzGU1vgw>



Link to the project repository: <https://github.com/alient12/DTPProject/>

Section 1 – Introduction

Industry 4.0 marks a transformative shift in manufacturing and industrial processes, driven by the integration of advanced technologies such as artificial intelligence, the Internet of Things (IoT), and cyber-physical systems. At the heart of this revolution is the concept of the digital twin (DT) - a dynamic, virtual replica of a physical asset or system. For autonomous robotic manipulators, digital twins play a crucial role in enabling predictive maintenance, real-time monitoring, and continuous performance optimization. By simulating various scenarios in a virtual environment, digital twins allow for safer and more efficient development, testing, and deployment of robotic systems, ultimately reducing downtime, enhancing adaptability, and supporting intelligent decision-making in complex and evolving industrial settings.

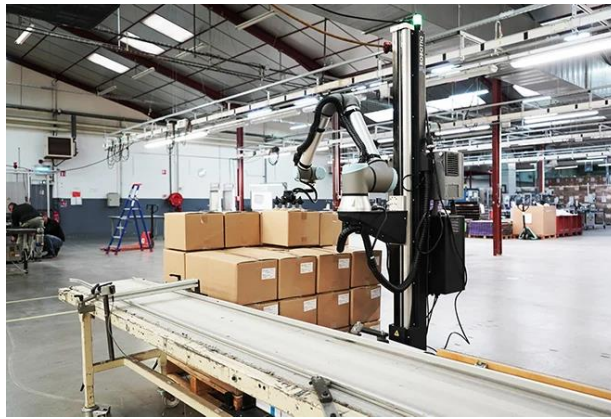


Figure 1: Pick and place task in an industrial environment

One of the common use cases of robotic manipulators is a pick and place task. An example of a robot performing such task is shown in Figure 1. Pick and place is a common and important task in robotics, where a robotic manipulator is used to grab objects from one location and move them to another. This process often involves several steps: detecting the object using a camera or sensor, planning the motion path, controlling the arm to reach and grasp the object, and then placing it accurately in a target position. Pick and place applications are widely used in industries like manufacturing, packaging, and logistics, where speed, precision, and repeatability are essential. Autonomous robotic manipulators performing these tasks can greatly increase productivity, reduce human labor, and maintain consistent quality in high-volume operations. At the same time, being able to monitor the manipulator robot performance, collect data on task completion rate and optimize the control policy is important from both the safety and business side. Being able to send commands remotely to the robotic cell is equally important.

In this project, we are building digital twin services for Kinova 3 with 7 degrees of freedom (Dof) robotic manipulator. For controlling the robot, a custom pick and place application was built based on ROS2 and libraries provided by Kinova. The positions (3D or joint space) of the robot are sent over the TCP/IP network to the developed web application for visualization and monitoring. For web applications we used the PyQT framework. The visualization of the robot movement, joint positions, and other monitoring features are implemented with the help of separate microservices. As digital twin services, we propose deviation monitoring, emergency stop, visualization and what-if simulation. To facilitate the emergency stop service, a dedicated ROS2 service was implemented as an enabler within the physical twin. An overview of the developed DT is illustrated in Figure 2.

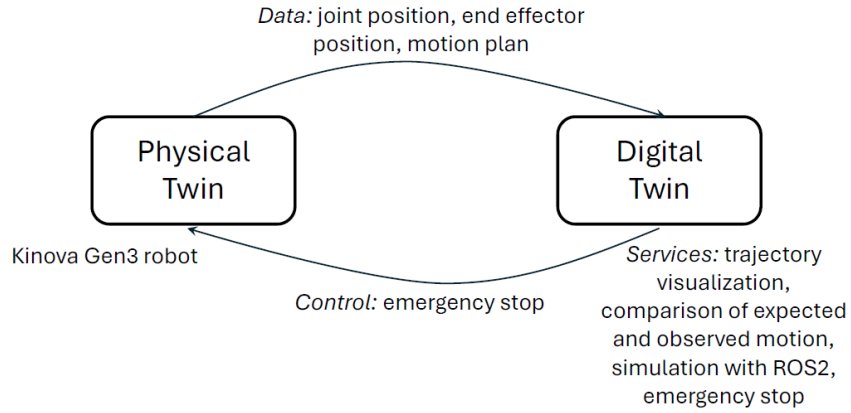


Figure 2: High level diagram of the developed digital twin

The figure illustrates the interaction between a physical and a digital twin of a Kinova Gen3 robot. The physical twin (PT) sends data such as joint positions, end effector position, and motion plans to the digital twin, which provides services like trajectory visualization, comparison of expected and observed motion, simulation using ROS2, and emergency stop functionality. In return, the digital twin can issue control commands, including an emergency stop, to the physical robot. In the next sections, we describe the implementation of PT and DT in more detail.

In Appendix, we present a detailed table describing the proposed DT characteristics.

Section 2 – Physical twin description

As the physical twin, we use a Kinova Gen3 7-DoF (degrees of freedom) robotic arm, which is part of the ROSIE mobile platform. The ROSIE platform consists of two Kinova robotic arms and a mobile base. In the future, we plan to extend the digital twin services to cover the entire ROSIE platform.

2.1 Physical twin overview

An illustration of the ROSIE platform is shown in Figure 3. In the scope of this project, the digital twin was developed for only one of the robotic arms (see left image in Figure 3).



Figure 3: ROSIE platform

A closer image of the PT is shown in Figure 4. The Kinova arm is equipped with Robotiq 2f-85 gripper [2], which is needed to perform the pick and place applications.



Figure 4: Physical twin performing a pick and place task

The physical twin runs a controller that instructs the robot to perform a pick-and-place task for selected objects. In the control code developed for this project, the robot executes a single pick-and-place action. It begins from a default (home) position, approaches the target, performs the pick, and places the object at a designated location. The object is then released, and the robotic arm returns to the home position.

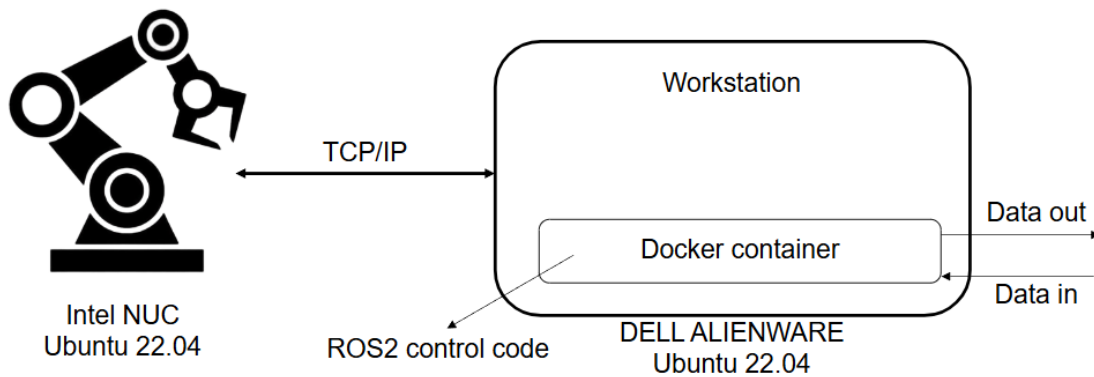


Figure 5: Physical twin architecture

The connection architecture of the PT is shown in Figure 5. The robotic arm is directly connected to an Intel NUC PC, running Ubuntu 22.04 OS. This PC is connected to a more powerful Dell Alienware workstation over a TCP/IP network (cable connection). To ensure better reproducibility and modularity of the developed robotic applications, all the control code is running in dedicated docker containers

of the workstations. Inside the container, a ROS2-based development environment is set up and the control code for the robot is hosted. To execute the control code on the robot, a corresponding command is issued to send data to the robot. An example of such command is shown below [3]:

```
ros2 launch kinova_gen3_7dof_robotiq_2f_85_moveit_config robot.launch.py \  
robot_ip:=192.168.42.191
```

The developed docker container is [available online](#).

2.2 Physical twin implementation

To control the robot, we chose to use ROS2 framework [4]. ROS 2 (Robot Operating System 2) is an open-source middleware framework designed to support the development of robust and scalable robotic systems. ROS 2 breaks the system into smaller, independent programs called nodes, each responsible for a specific function—like reading data from a sensor, planning movement, or sending commands to motors. These nodes communicate with each other using topics, which act like named channels for exchanging information. A node can publish messages to a topic when it has data to share, and other nodes can subscribe to that topic to receive the information in real time. This publish-subscribe model allows different parts of a robot to work together efficiently and makes it easier to develop, test, and update individual components without affecting the whole system.

For controlling the robot we used Moveit2 library [5]. It provides a suite of tools and libraries that enable robotic systems to plan, control, and simulate complex motion tasks such as pick-and-place, collision avoidance, and trajectory generation. It includes an Open Motion Planning Library (OMPL) [6], which is an open-source library that provides a collection of sampling-based motion planning algorithms for robotics.

As the first step for building our robotic application, we obtained the robot and gripper URDF models from the corresponding Kinova repository ([ros2 kortex/kortex_description/robots at main · Kinovarobotics/ros2 kortex](#)).

Based on this model, a so called “MoveIt configuration” can be generated. The primary function of “MoveIt config” is generating a Semantic Robot Description Format (SRDF) file for the robot. Additionally, it contains other necessary

configuration files for use with the MoveIt pipeline. This configuration will later be used by the motion planner to calculate the robot trajectory.

When controlling the robot, the control can be done either in joint or cartesian space. Joint space control involves sending commands directly to each of the robot's joints (e.g., joint angles, velocities, or torques). This approach is often simpler and more efficient, especially for low-level control tasks like trajectory following or motion planning based on predefined joint configurations. However, planning in joint space can be unintuitive, especially for tasks that require the end-effector to move precisely in the workspace (like reaching a point in space or following a surface).

In contrast, Cartesian space control focuses on commanding the motion of the robot's end-effector in the 3D workspace. Instead of thinking in terms of individual joints, the user or controller specifies a desired position and orientation (pose), velocity, or force to be applied in space. This is more intuitive for most manipulation tasks, especially in applications like assembly, insertion, or human-robot interaction.

However, Cartesian space control requires inverse kinematics (IK) calculations to convert desired end-effector poses into corresponding joint configurations. Given that the robot has 7 degrees of freedom (DoF) the inverse kinematics problem becomes redundant, meaning there are infinitely many joint configurations that can achieve the same end-effector pose. Because of this, from our experience, some joint configurations generated by the robot are suboptimal and take a longer path than needed. For instance, instead of making a simple move to the right, the robot might produce a trajectory to turn around 360 degrees before going to the target position. In order to prevent this, some complex planning constraint handling should be added. An example of such suboptimal trajectory is shown in the following video: <https://shorturl.at/fCeFM>.

Based on our initial experiments, we decided to control the robot in the joint space. We thus recorded the joint positions corresponding to specific robot poses for performing pick and place. In the control algorithm we set the robot to execute the obtained joint positions in the defined sequence. The demo of the robot executing the developed control algorithm is available at the following link: <https://shorturl.at/qDzA8>.

2.3 Implementation challenges

During the development of the controller for the described PT we encountered several challenges. We outline the main of them below.

1. Difficulties in setting up the controller

In our project we used ROS2, which is an updated version of ROS. Previously robotic manufacturers provided stable development packages for ROS. Given that ROS2 is more recent, some packages are still in development and there are stability issues. For instance, we had an issue setting up the environment according to the instructions from Kinova ros2 Kortex repository. After notifying the developers about it, they acknowledge that it existed (see Figure 6).

Hello Dmytro,

Thank you for reaching out. Here are the answers to your concerns:

1. [Issue 1](#)

The link to the realtime_tools Github repository was broken but now it is fixed and you shouldn't face issues when running the vcs import commands included in the README file

Figure 6: Installation issue confirmation from Kinova

Another challenge was related to the fact that Python library for Moveit2 framework is rather recent (Feb 2023) and a number of functionalities available for C++ implementation are not available in Python. At the same time, our WebSocket communication was implemented in Python, thus Python was the preferred choice for implementing the controller. At the same time, some issues arose when controlling the Kinova robot with python, regarding which we created the corresponding issue in Kinova repository:

[Controlling kinova gen 3 on ROS Humble with moveit2 built from the main branch](#)
· [Issue #274](#) · [Kinovarobotics/ros2_kortex](#)

2. Issues with gripper in simulation:

When simulating the gripper behavior, some issues were encountered, such as gripper not properly closing and opening, as shown in Figure 7.

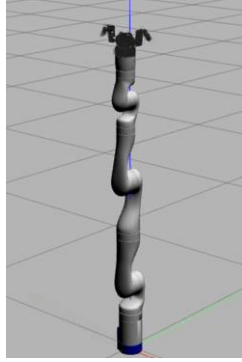


Figure 7: Issue with gripper in simulation

We communicated with Kinova and they confirmed the issue.

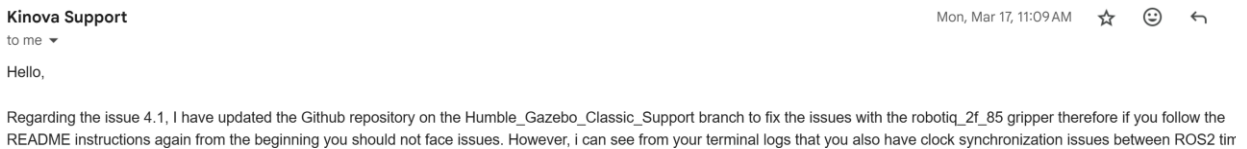


Figure 8: Kinova making changes in response to our request

3. Difficulties in Cartesian (3D) space control

Finally, as previously mentioned, controlling the robot in Cartesian space resulted in unpredictable and, at times, unsafe behavior during several experiments. As a result, we opted to use joint space control for the remainder of the work. Identifying the appropriate constraints and safeguards that would enable safe and reliable Cartesian space control remains an important direction for future work.

Section 3 – Digital twin architecture and services

In this section, we first provide a general overview of the implemented services within the DT. We then describe implementation details for each of the services and outline the related challenges.

3.1 Digital twin overview

The main goal of our DT is to provide insights to the engineering on the current operating state of the robotic arms, potential recommendations on the improvements and ways to increase efficiency as well as safety checking. We thus propose the following 5 services for the DT: visualization, singularity detection, simulation, deviation checking and emergency stop.

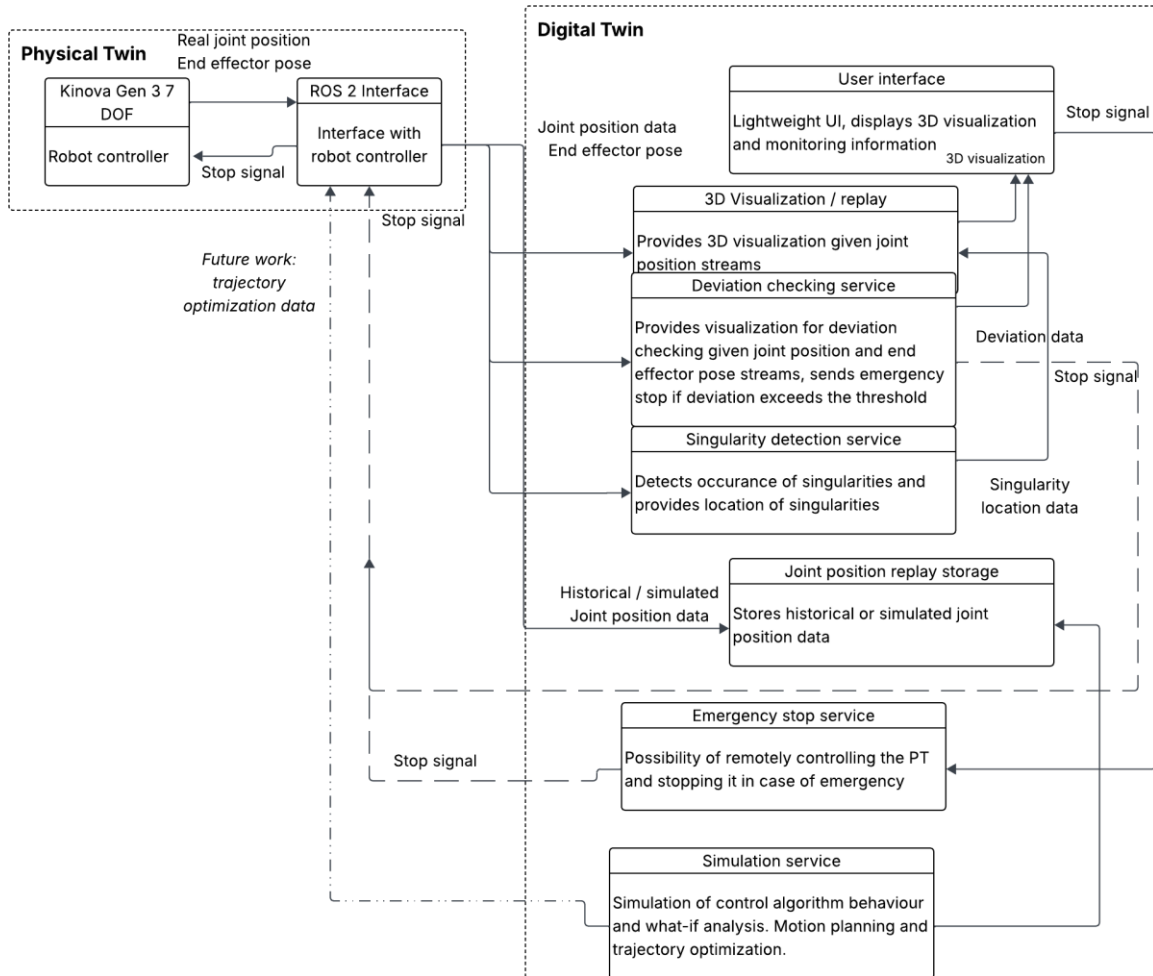


Figure 9: Overview of the proposed DT services and their relations
The overview of the proposed services and their relations are shown in Figure 9. The 3D Visualization service module plays a key role in the human interface layer. It

provides a 3D visualization of the robot's motion using the joint position and end effector (gripper) position streams. This module connects to a GUI that allows users to view the robot's status and performance visually. The visualization component also includes a deviation checking service, which compares commanded versus actual joint positions and end-effector poses. If discrepancies exceed a defined threshold, it can trigger an emergency stop service, safeguarding the robot and surrounding environment. Additionally, a singularity detection service monitors for occurrences of kinematic singularities and provides their locations to inform trajectory planning.

The joint position replay storage component supports both monitoring and development. It logs historical or simulated joint data, which can be replayed for offline visualization, debugging, or training of algorithms. This data repository also feeds into the visualization module and the simulation service, ensuring consistency across the digital twin components. It enables the minimization of the discrepancies between DT and PT behavior.

Finally, the simulation service facilitates the emulation of control algorithms, motion planning strategies, and what-if scenarios. It serves for testing different configurations, algorithms, or emergency conditions without engaging the real robot. As part of the future work, we plan to integrate the results of the simulations to the improvements in control logic on-line, during the robot operation.

3.2 DT services implementation description

In this subsection we discuss the implementation of the proposed services in more detail. The proposed digital twin constellation is composed of several key components, as illustrated in Figure 10. At the model and data level, the system incorporates both real-time and expected joint position information. A crucial part of this setup is the Unified Robot Description Format (URDF) model, which defines robot's physical structure, including its links, joints, sensors, and visual or collision properties. The URDF is an XML-based format standardized by the Robot Operating System (ROS) and it is widely used across simulation platforms such as Gazebo and Robotics Toolbox.

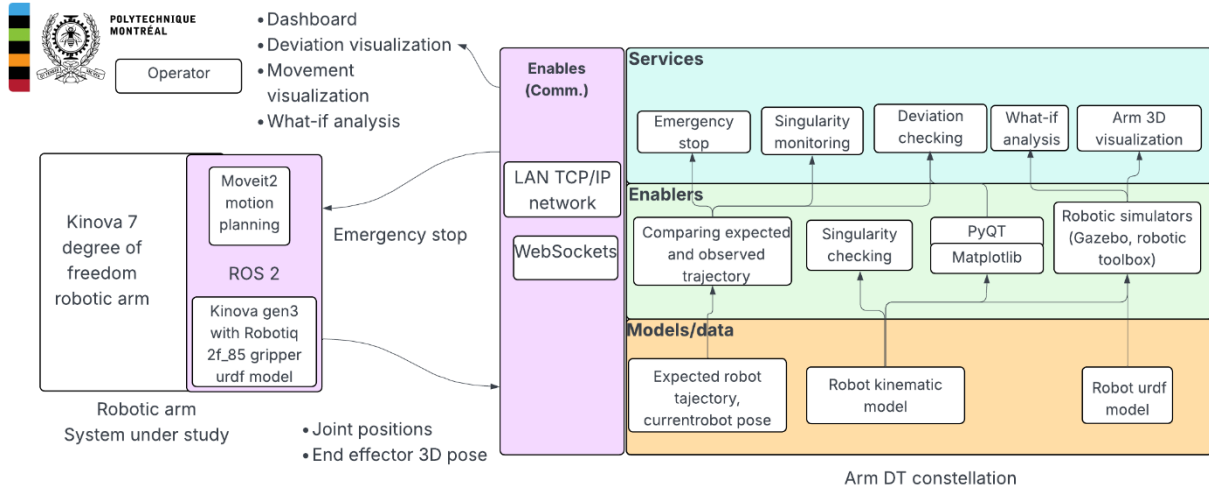


Figure 10: Digital twin constellation

The robot's kinematic model mathematically describes how joint movements affect the position and orientation of the end-effector in space. This model includes both forward kinematics, which computes the end-effector pose from joint values, and inverse kinematics, which determines the required joint configurations to reach a desired pose. Together, the URDF and kinematic models form the foundation for robotic motion planning and control.

For this project, we utilize a URDF model to represent the Kinova Gen3 robot, which we sourced from an [open-source repository](#). We verified the integrity of this model using ROS's built-in URDF tools to ensure accuracy and compatibility. The model is then employed by the Robotics Toolbox [7] simulation enabler to define the kinematic structure and joint behavior of the robot within our digital twin. Robotics Toolbox is used to provide a 3D visual simulation of the robot, which aids in analysis and monitoring. In addition to the simulation enabler, we have implemented a deviation checking enabler. This module monitors discrepancies between the real-time and expected end-effector trajectories, based on joint position data. It generates a 3D trajectory graph of the end-effector, which is displayed in the graphical user interface (GUI). This visualization is intended to assist the operator in making informed decisions, particularly when determining whether to activate the emergency stop feature. In addition, we have added a singularity estimation enabler that process the current joint positions and determines if the robot is in the singularity state, which can be risky in production environments [8].

The GUI components are developed using the PyQt framework for the interface, and Matplotlib for graphical rendering. While Matplotlib is responsible for all graph-related displays except the 3D visualization, PyQt manages the layout and interaction capabilities of the user interface. Together, these tools create an intuitive and responsive environment for interacting with the digital twin system.

Underneath, the digital twin is architected as a constellation of microservices that work in union to provide the described DT services. Specifically, the architecture includes a microservice for the graphical user interface (GUI), a microservice for robot simulation and visualization, and a microservice for communication with the physical twin. This modular design promotes scalability, maintainability, and parallel execution. All microservices are orchestrated by a central handler, which is responsible for launching them as independent processes, as shown in Figure 11. To facilitate data exchange between components, particularly information related to the physical twin—the handler sets up shared queues, which serve as the primary means of inter-process communication. This enables efficient and structured data flow while maintaining separation of concerns between the services. The simulation microservice is responsible for rendering the 3D model of the robot and delivering it as an interactive environment accessible via an HTTP connection. This allows seamless integration with the GUI, which uses the visualization to provide the operator with a real-time view of the robot’s state. In addition to monitoring, the GUI supports a lightweight what-if analysis, enabling the operator to explore different scenarios and assess the potential impact of deviations or failures in a simulated environment.

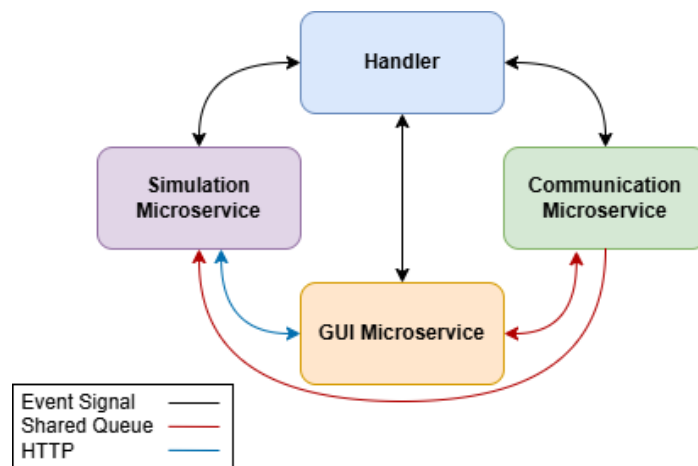


Figure 11: DT services and inter-service communications

Additionally, we implemented a microservice orchestrator to ensure seamless coordination between individual services. The orchestrator launches each microservice in a separate CPU thread, enabling parallel execution and efficient resource usage. It also manages shared queues and events, which are passed to service instances to facilitate inter-service communication. When the GUI is closed, the orchestrator safely shuts down the system by clearing all queues and gracefully terminating each service, ensuring a clean and controlled shutdown of the digital twin

3.2.1 Simulation service

Another important DT service is simulation. We provide two modes of simulation service with different levels of simulation complexity: advanced simulation with physics-engine based simulators and simplified simulation based on robot kinematic model and visualization. The advanced simulation enables in-depth testing of complex scenarios, incorporating data from multiple sensors and interactions with objects of different shapes and weights. In contrast, the lightweight simulation mode allows for rapid prototyping and visualization of robot movements. The simulation service can operate independently or in coordination with the physical twin, allowing for parallel experimentation, regression testing, and what-if scenario analysis. It is particularly useful for validating motion plans or new control strategies before deploying them to the real robot, thereby reducing the risk of hardware damage and accelerating development cycles. Currently, the simulation is done independently of PT operation. As part of the future work, we plan to integrate on-line PT reconfiguration with optimized motion plans from simulation.

Advanced simulation

The advanced simulation service is built based on Gazebo robotics simulator and ROS 2. Gazebo supports multiple physics engines (such as ODE, Bullet, and DART), enabling accurate simulation of dynamics, collisions, and contacts. Gazebo includes support for a wide range of sensors like cameras, lidars, IMUs, and GPS, allowing users to generate rich synthetic data. Moreover, it integrates easily with ROS 2 facilitating robotic software development and testing. An illustration of a Kinova Gen3 in the simulated Gazebo environment is shown in Figure 12,

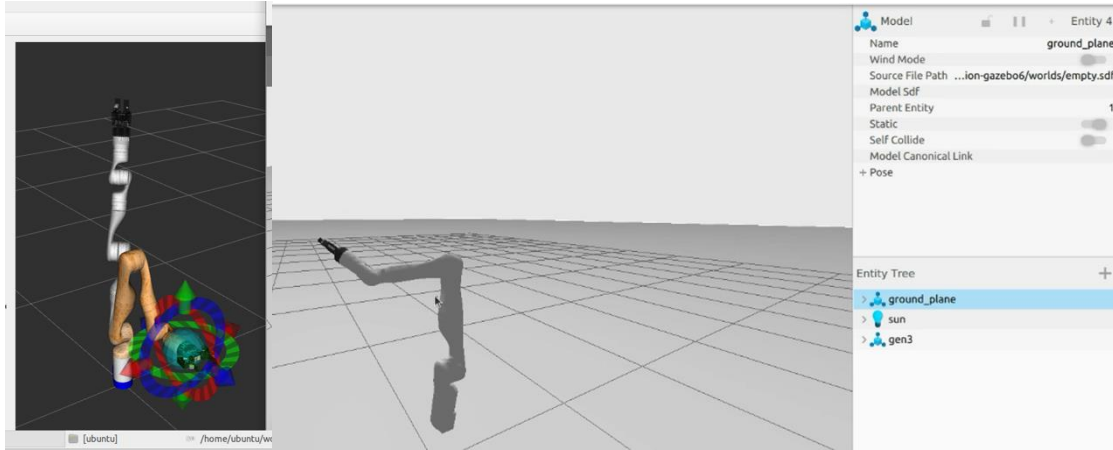


Figure 12: PT simulation in Gazebo environment

Lightweight simulation

For lightweight simulation, we use the Robotics Toolbox for Python (RTB), a simulator tailored for robotic arms that includes a built-in visualizer called Swift. While Gazebo provides advanced simulation capabilities, RTB is well-suited for simpler tasks such as solving kinematic and dynamic equations and performing basic collision detection. These features are sufficient for our visualization needs and enable rapid prototyping while keeping the setup platform-agnostic and easy to install. This lightweight approach is a key differentiator from traditional physics-based simulators, which often come with demanding system and hardware requirements.

Implementation

As mentioned previously, integration of Gazebo was straightforward, once the ROS2 based controller is developed. The integration of RBT required more effort in the beginning. At the earliest stage of the project, we started searching for open-source projects that provide robotic arms visualization features such as RTB to serve as the foundation. First, we generated a stream of random trajectories to verify this functionality of RTB. This feature can also be used to replay prior movements if historical data is available. After confirming the feasibility of RTB for our project, we added a feature to RTB to change the robotic arm color and transparency to facilitate the singularity snapshots. In order to add such a functionality to the code, we had to define a specific class for our robot which derives the original Kinova Gen3 class from RTB library. For visualization purposes, we added a feature that allows us to take a snapshot of the robot when a particular movement is executed.

This information will allow the operator to debug and remedy the issue in the designed trajectory. Figure 13 shows an example of such snapshot.

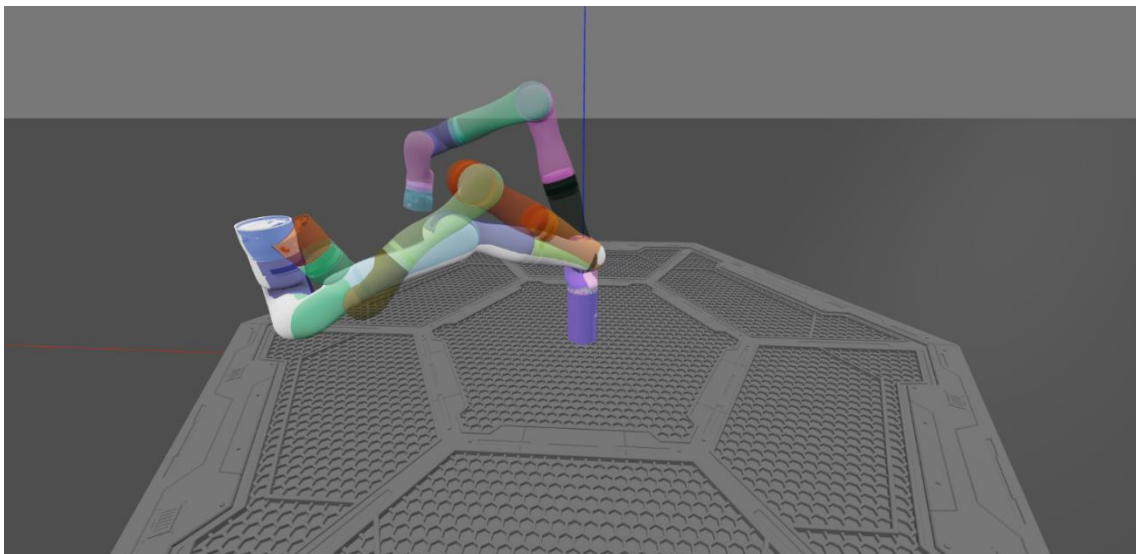


Figure 13: Multiple snapshots of the robot arm in different positions and colors

Additionally, we modified the Swift visualization code to support integration with our custom GUI over an HTTP connection, as the original Swift implementation was limited to launching visualizations in the default browser. With our improvements, Swift can now accept an argument that prevents it from opening a browser, allowing external GUIs to embed and utilize the visualization service seamlessly. Once the Swift environment and RTB were properly configured, we built a simulator service class on top of the RTB library. This class abstracts the complexity of RTB's API, providing a simplified interface for simulation control. Simulation parameters are passed through the class constructor, enabling object-oriented usage and the creation of multiple simulator instances with different configurations. The simulator runs in a loop for each simulation step and supports two modes: offline and online. In offline mode, the simulator uses RTB for both robot control and visualization, while in online mode, it receives joint positions from an external communication service and uses RTB solely for visualization.

Challenges

In the very beginning of the project, we realized that the Swift code is no longer being supported by the original developer. First, we faced a bug which was causing the environment to crash, which hopefully will be fixed by GitHub contributor

Antun Skuric¹. The Swift code used for robot visualization presents another challenge. It creates a local server which only serves the first client correctly. At the beginning of the project, when we were verifying the feasibility of basing our project on RTB, we observed that Swift opens a browser to show the 3D visualization from a local server. We concluded that we can later copy the server address and use it in our GUI using PyQt WebKit or electron which allows developers to load a webpage as a GUI. Later during the development of the GUI, we realized that the server sends an empty to environment without any robot to our GUI. It took about two weeks to find a solution to start the server without opening a browser, so our GUI would be the first client for the Swift environment, and it can receive the environment with a robotic arm inside it. We also added this change to open source, so other developers can use it².

3.2.2 Singularity detection service

In robotic manipulators, a singularity refers to a specific configuration where the robot loses some ability to control the direction or speed of its end-effector. This happens when the robot's joints align in a way that causes the mathematical model—specifically, the Jacobian matrix—to lose rank. As a result, certain movements become impossible or extremely sensitive: small changes in joint angles can cause large and unpredictable motions of the robot's end-effector. For example, if a robotic arm extends fully in a straight line, it may not be able to move sideways or apply force in certain directions. These situations can lead to instability or errors in tasks like path planning and control. That's why monitoring for singularities is essential—it helps ensure smooth and predictable robot behavior by allowing the system to avoid or compensate for these problematic configurations.

Overview

Withing the Robotic toolbox, in the new Kinova robot class, we added a method which determines if the robotic arm has entered a state of singularity. The singularity detection service identifies robot position singularities. These singularities occur when the robot's joint positions create an unsuitable state for the robot arm, often due to division by zero. This can result in abrupt movements of the robot arm, posing risks to both the surrounding environment and the robot itself. This service is called in each timestep of the simulation to ensure the safety of the environment. The effect

¹ [bugfix - do not update the deleted items on env.step\(\) by askuric · Pull Request #49 · jhavl/swift](#)

² [Qt5/Qt6 integration · Issue #53 · jhavl/swift](#)

of this feature can be seen in Figure 14. The right arm is the current position of the robotic arm while the left arm is a snapshot of the past position which is transparent. The links and joints that were involved in singularity are shown in red.

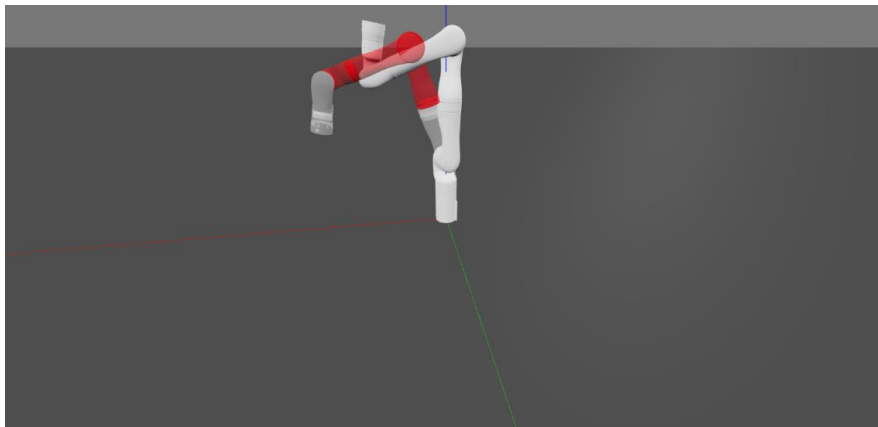


Figure 14: singularity checking service in action with snapshot enabled

Implementation

This service calculates the Jacobian matrix of the joint positions, followed by calculating the condition number which is used for the detection of general singularity. Alternatively, the rank calculation of each parts of Jacobian matrix helps to identify specific singularity such as wrist, shoulder and elbow singularities.

3.2.3 Visualization service

The developed visualization service allows operators to visually track the robot's movement, monitor joint positions, and observe the end-effector's pose within the workspace. This helps in quickly identifying deviations from expected trajectories, enabling faster diagnostics and making more informed decisions on the robot control. For instance, by observing the dashboard the operator might decide that the robot state is risky and issue an emergency stop request. Moreover, this visualization can be useful for debugging if some issues occur with the robot in the production environment.

Overview

For the development of the GUI, we decided to use PyQt5 framework as it is a popular framework with great features and being cross-platform. The GUI consists of four main parts (Figure 15):

- 1- 3D visualization
- 3- Joints plots

- 2- End-effector 3D plot
- 4- Dashboard

The 3D visualization of the robot utilizes the simulation service to produce a photorealistic visual effect of the current state of the robot arm in 3D. The joints plots and end-effector 3D plot are powered by Matplotlib library with PyQt backend. While the joints plots simply show the robot arm joints in real-time, the end-effector 3D plot shows the robot end-effector position in real-time and a few previous timeframes which let the user see the trajectory that the robot end-effector is following. This also allows the user to identify any significant deviation of the end-effector to the intended trajectory.

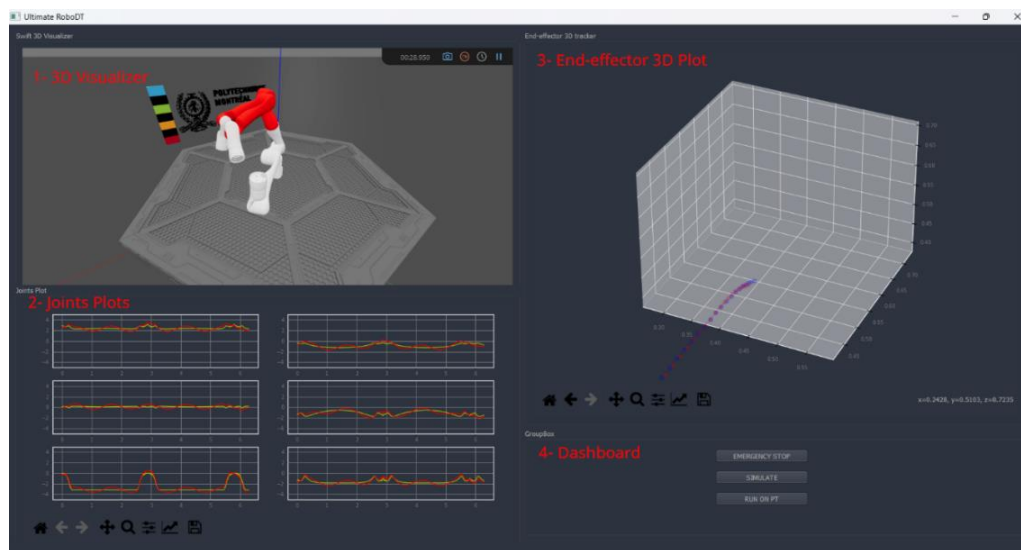


Figure 15: Digital twin GUI

Implementation

The 3D visualization of the robot is done by simulator service and is being streamed over an HTTP connection which is connected to the GUI using a PyQt5 WebKit widget as server client. The plots are being controlled by a separate CPU thread to read the latest data sent by physical twin from a shared queue.

Challenges

The most challenging aspect of developing this GUI was designing clean and efficient code capable of handling six plots, a trajectory graph, and 3D arm visualization without causing crashes. Significant effort was dedicated to code optimization, allowing the interface to run smoothly with minimal resource usage.

3.2.4 Deviation monitoring service

The deviation monitoring service is important in ensuring the reliability and safety of robotic manipulator operations. By continuously comparing the actual joint positions and end-effector pose against the expected trajectory, the system can quickly detect anomalies or unintended behavior. This is especially important in tasks requiring high precision or operating in dynamic environments, where even small deviations can lead to failure or damage.

Implementation

The trajectory of the end effector is also specifically visualized in the GUI. This is important in order to provide the operator with an overtime view of the movement of the arm and the end effector. The trajectory visualization service receives expected and real-time joint data of the end effector from the shared queue, then adds each subsequent time-series datapoint to the 3D graph. We then applied a fading mechanism that deleted the oldest points in the sequence overtime. The resulting plot then displays only recent end effector movement and its trajectory. This service is integrated into the greater GUI service and therefore part of the monitoring services which enables our human-in-the-loop to make decisions such as triggering the emergency stop.

Challenges

During our design process, it is discovered that if the trajectory graph contained all joints, it would be confusing and overwhelming to the operator and its utility would be reduced. Thus, we elected to only display the trajectory of the end effector.

3.2.5 Emergency stop

Overview

The emergency stop function is the part of the DT that completes the feedback loop to the PT and elevates our DT from being just a digital shadow. All monitoring services provided previously are crucial to allow the operator to visualize and activate the emergency stop when appropriate. The emergency stop function has 3 components: the GUI emergency stop on the DT side, emergency stop event as implemented in the DT and the ROS 2 acting component on the PT side. Moreover, the emergency stop maybe launched automatically by a deviation checking service, if the robot trajectory deviation exceeds a defined threshold.

Implementation

Implementing the emergency stop functionality required not only modifications to the digital twin (DT) but also the addition of components on the physical twin (PT) side. We developed an additional ROS 2 service on the PT, which receives the emergency stop signal from the DT and forwards a stop command to the Kinova robot. On the DT side, a dedicated emergency stop button was integrated into the GUI for the operator. When activated, the GUI triggers a shared emergency stop event that propagates through the DT and ultimately communicates the stop request to the ROS 2 service. This setup ensures a coordinated and reliable response across both the digital and physical systems.

Challenges

A major challenge was ensuring reliable two-way communication between the digital twin (DT) and the physical twin (PT). While receiving data from the PT to the DT was already in place, implementing the emergency stop functionality required adding a robust mechanism for transmitting messages from the DT back to the PT. To achieve this, we designed a WebSocket based communication protocol running at 10 Hz. In each communication cycle, the PT first sends data to the DT and waits for an acknowledgment. Once received, the DT responds by sending the current value of the emergency stop flag, which the PT then acknowledges. This communication pattern repeats in a continuous loop. Based on our experience, communication at frequencies higher than 10 Hz proved unreliable, with some packets being lost during transmission, resulting in unnatural robot movement visualization.

3.4 Digital to physical twin communication

The communication between the Digital Twin (DT) and the Physical Twin (PT) was established using a WebSocket protocol, enabling real-time, bidirectional data exchange over a persistent connection. The PT continuously streamed its joint positions, end-effector pose, and current motion plan to the DT, allowing the digital model to stay synchronized with the physical system's state. This data was used to visualize and monitor the robot's behavior in real time. In the opposite direction, the DT had the capability to issue critical control commands, such as an emergency stop signal, which was sent to the PT over the same WebSocket connection to ensure a

fast and reliable response when safety conditions required immediate intervention. Figure 16 shows the path for sending data packets from the DT to the actual robot.

Challenges

The main challenge was the delay we encountered during the test on the real robot. Before testing on the real robot, we tested the communication code on a stream of mock data to ensure correct functionality. In order for the DT to communicate with the PT, the DT hardware needs to be connected to a Wi-Fi router which the robot uses to communicate with ROS. All three devices are connected to this router provided by the robotics lab, and we cannot modify this configuration. During our tests, we observed that the network connection was slow, which was confirmed by an increased latency of web search.

Another challenge was synchronization of data coming from ROS2 topics and the rate of sending the data back to the DT via WebSockets. Experimentally, we established a working frequency of 10 Hz, which might not be sufficient for some latency sensitive applications.

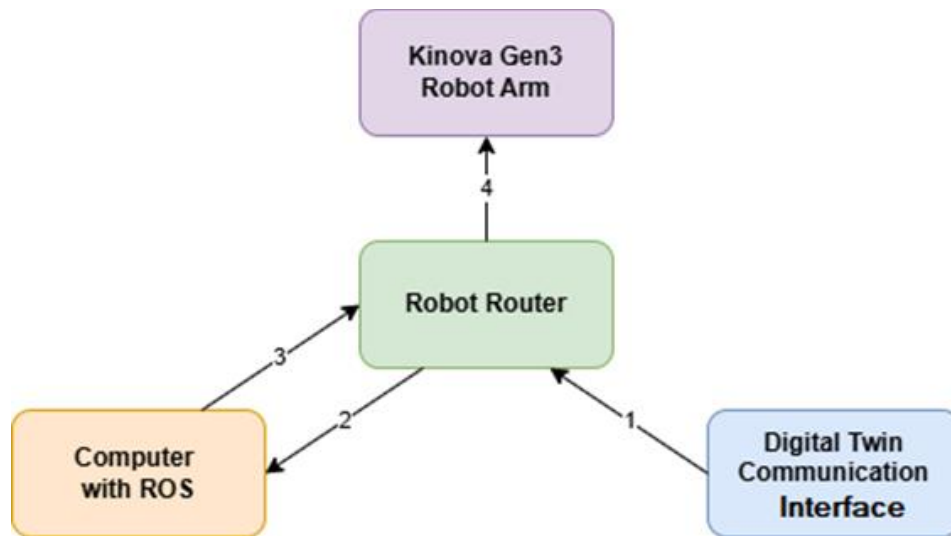


Figure 16: Communication between DT and PT

As a future improvement, instead of relying on WebSocket-based communication, the data can be transmitted directly to the digital twin using ROS 2. This would require running ROS2 nodes on the DT side, which complicates the DT setup. At the same time, this setup would make communication more robust and scalable. New data could be added simply by creating a new topic and subscribing DT to it. In some sense it is similar to the RabbitMQ style communication, but better adopted to the robotics domain.

Conclusions

In this project, we demonstrated a proof of concept for a flexible, platform-agnostic, and lightweight digital twin tailored for Industry 4.0 applications by developing a digital twin of a ROS 2-based collaborative robotic manipulator, the Kinova Gen3. We have successfully implemented five core DT services: visualization, simulation, emergency stop, singularity monitoring, and trajectory deviation monitoring. Our system aligns with the core principles of a digital twin, as it receives real-time pose data from the physical twin (PT) and maintains the capability to send control commands, such as emergency stop signals, back to the PT. As part of future work, we aim to enhance the communication protocol to support more real-time interaction. A promising direction involves using ROS 2 not only for controlling the PT but also for DT-PT communication, replacing the current WebSocket-based approach. Another valuable avenue is leveraging simulation for online PT reconfiguration and motion plan optimization. Another promising avenue of work is leveraging simulation for online PT reconfiguration and motion plan optimization.

References

- [1] Kinova Robotics, "Gen3 robot," [Online]. Available: <https://www.kinovarobotics.com/product/gen3-robots>. [Accessed: Apr. 18, 2025].
- [2] Robotiq, "Adaptive Grippers – Two-Finger Gripper," [Online]. Available: <https://robotiq.com/products/adaptive-grippers#Two-Finger-Gripper>. [Accessed: Apr. 18, 2025].
- [3] Kinova Robotics, `ros2_kortex`, GitHub repository, https://github.com/Kinovarobotics/ros2_kortex, accessed Apr. 18, 2025.
- [4] Open Robotics, ROS 2 Documentation: Humble Hawksbill, <https://docs.ros.org/en/humble/index.html>, accessed Apr. 18, 2025.
- [5] PickNik Robotics, MoveIt Documentation, <https://moveit.picknik.ai/main/index.html>, accessed Apr. 18, 2025.
- [6] Sucan, I. A., Moll, M., & Kavraki, L. E. (2012). The open motion planning library. *IEEE Robotics & Automation Magazine*, 19(4), 72-82.
- [7] Corke, P., & Haviland, J. (2021, May). Not your grandmother's toolbox—the robotics toolbox reinvented for python. In 2021 IEEE international conference on robotics and automation (ICRA) (pp. 11357-11363). IEEE.
- [8] Xu, W., Zhang, J., Liang, B., & Li, B. (2015). Singularity analysis and avoidance for robot manipulators with nonspherical wrists. *IEEE Transactions on Industrial Electronics*, 63(1), 277-290.

Appendix

Proposed DT characteristics:

DT Characteristic	Description	
MC1: System-under-Study	Describes the SUS, i.e., the PT, of the system of interest.	The PT is a 7-Dof Kinova 3 robotic arm programmed to perform a pick and place task. It is controlled by a computer running ROS2. The PT is accessible through a Wi-Fi router to external devices.
MC2: Physical acting components	Mechanisms the DT can use to act on the PT.	Emergency stop service implemented in ROS 2 allow the DT to communicate to the PT to stop any movement of the robot.
MC3: Physical sensing components	Mechanisms the PT can use to transfer data to the DT.	Joint position sensor: this data is provided by the robot controller as sampled from sensors located in the physical joints at 1kHz +. Expected Joint position: this data is provided by the robot controller.
MC4: Physical-to-Virtual Interaction	Data transmitted from PT to DT, including inputs/events the DT processes.	Joint position data, a time series data of the location of all 7 joints present on the robot, provided by sensors embedded in the robot at a frequency of 1kHz. Expected Joint position data, a time series data of the expected location of the 7 joints at the present time. This is provided through ROS2 at a frequency of ~100 Hz. Data is transmitted over WLAN networks using WebSocket. The PT provides this data through the ROS 2 interface.
MC5: Virtual-to-Physical Interaction	Data transmitted from DT to PT, including output commands generated by the DT.	Emergency stop signal The Emergency stop signal, upon activation by the operator, is transmitted over WLAN using WebSocket to the ROS 2 interface, which is then transmitted to the PT.

MC6: Digital Twin Services	Services like optimization, task planning, visualization, etc., that the DT provides.	Visualization Simulation Deviation detection Singularity detection What-if analysis
MC7: Twinning Time-scale	Time rates and synchronization frequency between DT and PT.	The DT synchronizes with the PT in real time with an update frequency of 60 Hz in ideal conditions. The theoretical twinning frequency is determined by the rate which the sensors output data on the PT. The visualization updates at ~10 Hz
MC8: Multiplicities	Composition and hierarchy of DTs—whether centralized or decentralized.	No multiplicity within each DT. Multiple DTs may exist corresponding to multiple arms.
MC9: Life-cycle stages	Life-cycle phases the DT supports (e.g., design, manufacturing, operation).	Supports operational planning and operational stages by providing what-if analysis and real time monitoring.
MC10: Digital Twin Models and Data	Models (behavioral, structural, data) and their role in the DT constellation.	URDF model for robot joint movement simulation and trajectory planning. 7 DoF robot kinematic model
MC11: Tooling and Enablers	Software tools and enablers that support DT services.	Software packages used in this DT include PyQT library for GUI implementation, Matplotlib for graph displays and Robotic toolbox for robot visualization using URDF model. Communication between the SUS and the DT is established through WebSocket over WLAN.
MC12: Digital Twin Constellation	The orchestration of DT models, tools, services, and interactions.	The DT is orchestrated as defined in the source code. Currently no alternative configuration options are implemented.
MC13: Twinning Process and Digital Twin Evolution	DT development timeline, methodology, and milestones.	This project has 3 milestones: Implementation of the simulation service, Implementation of the UI and finally implementation of the communication

		interface between the SUS and the DT including the emergency stop service.
MC14: Fidelity and Validity Considerations	Accuracy, uncertainty, validation mechanisms of the DT models.	The DT receives joint position sensor data and expected trajectory data over WebSocket at a rate of 10Hz. The inaccuracy of this data is not expected to exceed 1%.
MC15: Digital Twin Technical Connection	Technical details of network and communication between DT and PT.	Upon initialization, the DT listens for data provided by the ROS 2 interface through WebSocket.
MC16: Digital Twin Hosting/Deployment	Hosting environment of the DT (e.g., cloud-based, local network).	The DT is hosted locally within the same WLAN network as the SUS.
MC17: Insights and Decision Making	Indirect DT outputs like semantic insights and decision recommendations.	DT user interface provides insights such as past joint singularities detected and the robot state at time of detection, provides information such as individual joint velocity, expected vs real trajectory.
MC18: Horizontal Integration	Integration with other systems beyond the DT (e.g., enterprise systems, databases).	The DT contains infrastructure for replaying historical or simulated joint positions, integration with external databases is possible.
MC19: Data Ownership and Privacy	Ethical and legal concerns around data control and user privacy.	Data is stored locally and does not involve any third party unless the user explicitly implements such features.
MC20: Standardization	Compliance with DT-related standards (e.g., AAS, FMI).	N/A
MC21: Security and Safety Considerations	Risks and safeguards around remote DT operation, cybersecurity, and human safety.	Security aspect inherited from the implementation of the WebSocket library in python.